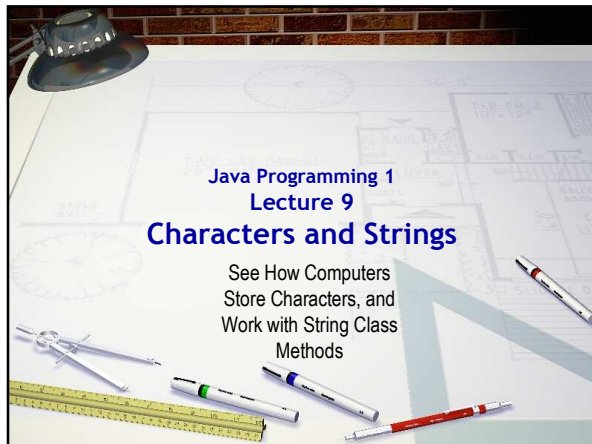
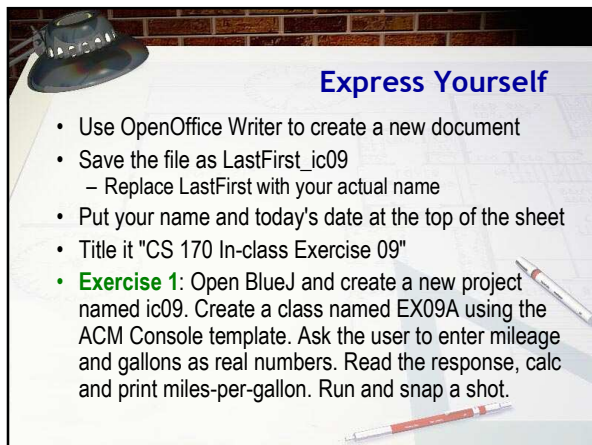
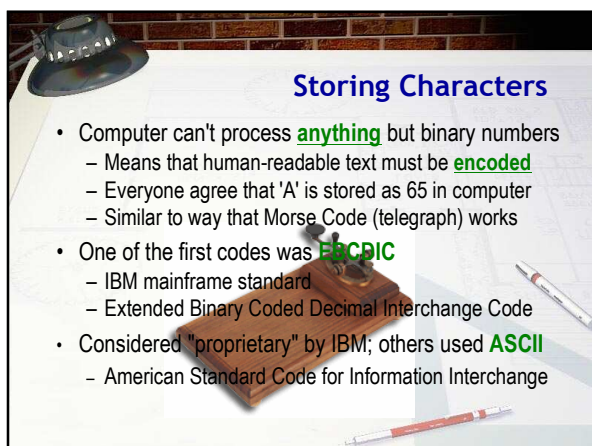








Unicode

- ASCII was designed to be efficient
 - Most characters stored in 7-bits
 - Small size meant quick transmission
- Modifying it for non-English languages was difficult
 - The "extra bit" used to provide alternate characters
 - "Code-pages" allowed DOS programs to switch
- Better solution is to use more memory than a byte
 - Several different variations
 - Most of the world has standardized on Unicode
 - Go visit at www.unicode.org

Dec	Hex	Char	Dec	Hex	Char
32	20	Space	64	40	@
33	21	!	65	41	A
34	22	"	66	42	B
35	23	#	67	43	C
36	24	\$	68	44	D
37	25	%	69	45	E
38	26	&	70	46	F
39	27	'	71	47	G

The char Data Type

- The **char** data type is used to store **single** characters
 - Uses two bytes of storage per character
 - Each storage location is interpreted using Unicode
- You create **char** variables just like other primitives

```

char initial    = <char literal>;
char controlA  = <char literal>;
char hTab      = <char literal>;
char backslash = <char literal>;
char copyright = <char literal>;

```

Writing char Literals

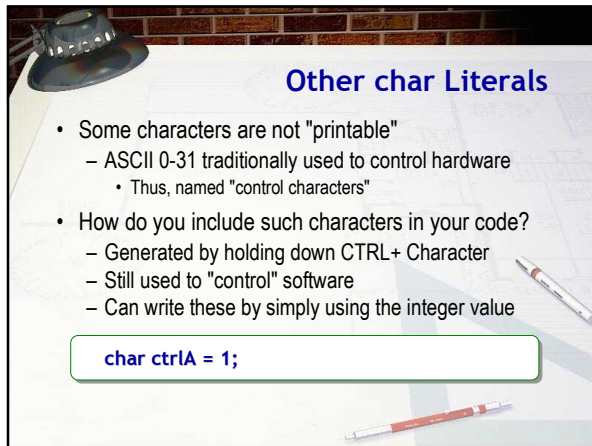
- How do you write a literal D? Here's a first attempt:


```
char initial = D;
```

 - Can't just use the character; looks like a variable
 - Must use some **punctuation**
 - Character used in Java is a single quote (')

```
char initial = 'D';
```

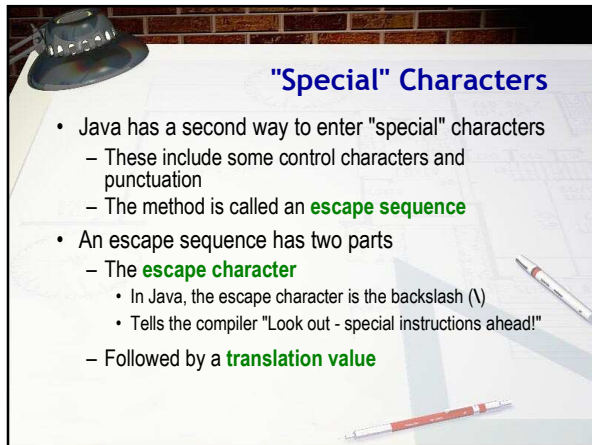
 - Single/double quotes are **not interchangeable**



Other char Literals

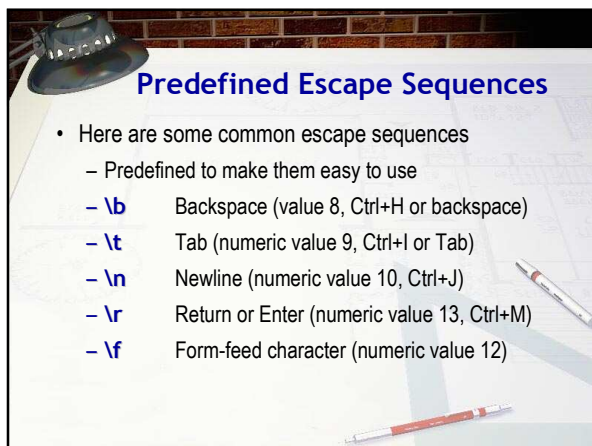
- Some characters are not "printable"
 - ASCII 0-31 traditionally used to control hardware
 - Thus, named "control characters"
- How do you include such characters in your code?
 - Generated by holding down CTRL+ Character
 - Still used to "control" software
 - Can write these by simply using the integer value

```
char ctrlA = 1;
```



"Special" Characters

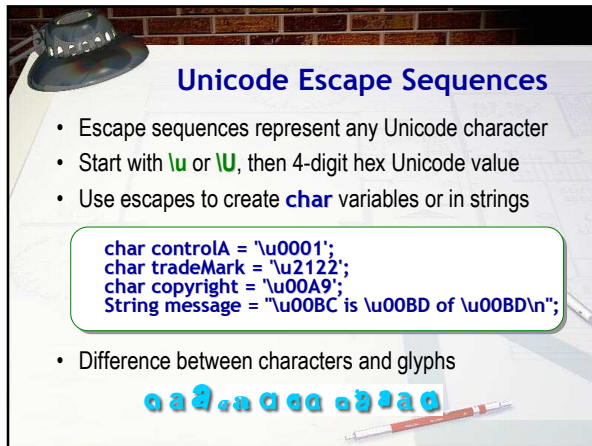
- Java has a second way to enter "special" characters
 - These include some control characters and punctuation
 - The method is called an **escape sequence**
- An escape sequence has two parts
 - The **escape character**
 - In Java, the escape character is the backslash (\)
 - Tells the compiler "Look out - special instructions ahead!"
 - Followed by a **translation value**



Predefined Escape Sequences

- Here are some common escape sequences
 - Predefined to make them easy to use

\b	Backspace (value 8, Ctrl+H or backspace)
\t	Tab (numeric value 9, Ctrl+I or Tab)
\n	Newline (numeric value 10, Ctrl+J)
\r	Return or Enter (numeric value 13, Ctrl+M)
\f	Form-feed character (numeric value 12)



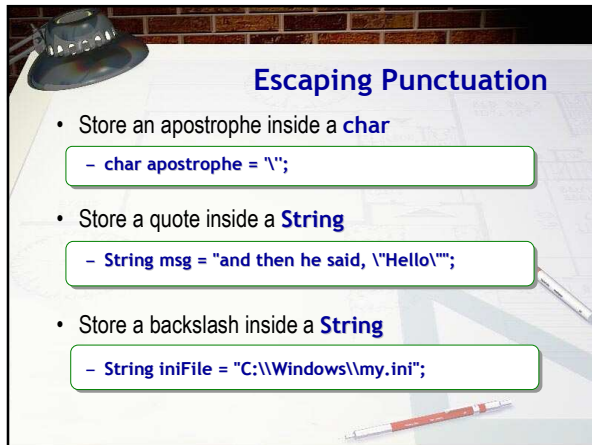
Unicode Escape Sequences

- Escape sequences represent any Unicode character
- Start with `\u` or `\U`, then 4-digit hex Unicode value
- Use escapes to create **char** variables or in strings

```
char controlA = '\u0001';  
char tradeMark = '\u2122';  
char copyright = '\u00A9';  
String message = "\u00BC is \u00BD of \u00BD\n";
```

- Difference between characters and glyphs





Escaping Punctuation

- Store an apostrophe inside a **char**

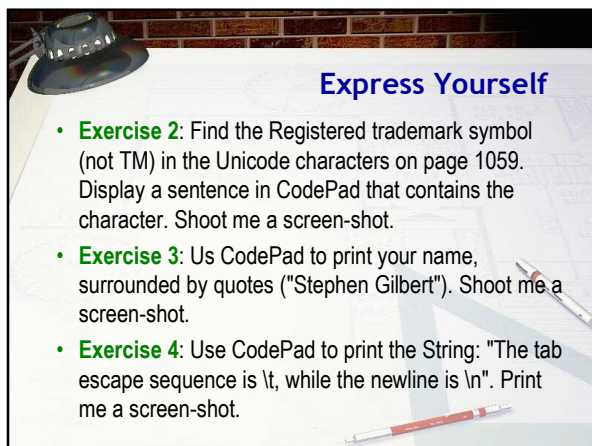
```
- char apostrophe = '\'';
```

- Store a quote inside a **String**

```
- String msg = "and then he said, \"Hello\"";
```

- Store a backslash inside a **String**

```
- String iniFile = "C:\\Windows\\my.ini";
```

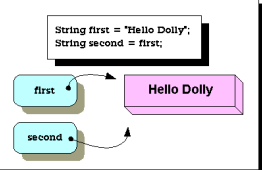


Express Yourself

- **Exercise 2:** Find the Registered trademark symbol (not TM) in the Unicode characters on page 1059. Display a sentence in CodePad that contains the character. Shoot me a screen-shot.
- **Exercise 3:** Use CodePad to print your name, surrounded by quotes ("Stephen Gilbert"). Shoot me a screen-shot.
- **Exercise 4:** Use CodePad to print the String: "The tab escape sequence is `\t`, while the newline is `\n`". Print me a screen-shot.

Meet the Strings

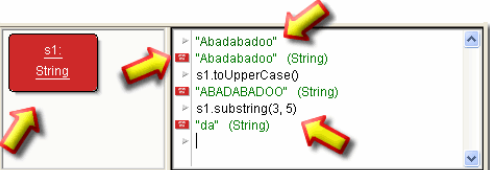
- What are **Strings** in Java?
 - **Immutable** sequences of 0...n characters
- **Strings** are like primitives in several ways
 - Initialize using a literal rather than using **new**
- **Strings really** are objects, though, not primitives
 - Strings are references not values



The diagram shows two variables, 'first' and 'second', each in a light blue box. Arrows from both boxes point to a single pink box labeled 'Hello Dolly'. Above the boxes, a code snippet reads: 'String first = "Hello Dolly";' and 'String second = first;'. This illustrates that both references point to the same String object in memory.

Strings and BlueJ

- Code Pad makes it easy to work with **String** methods
 - Type a **String** literal into Code Pad
 - Drag it and drop it on the Object Bench
 - Use the object to experiment with **String** methods



The screenshot shows the BlueJ IDE. On the left, the 'Object Bench' contains a red box labeled 's1: String'. On the right, the 'Code Pad' contains several lines of code with red arrows pointing from the 's1' object to the code: '> "Abadabadoo" (String)', '> "Abadabadoo" (String)', '> s1.toUpperCase() (String)', '> "ABADABADOO" (String)', '> s1.substring(3, 5) (String)', and '> "da" (String)'. This demonstrates how to use a String object to call methods.

A Tale of Two Strings

- The **null String** is an **uninitialized String**
 - It's reference contains the value **null**
 - The **only thing** you can do to it is initialize it
- The **empty String** is a **String** of length 0
 - Send messages to the empty **String**, but not the **null**

```
String emptyString = "", nullString;  
int len = emptyString.length(); // OK  
int len2 = nullString.length(); // Error
```

String Operations

- Java "overloads" two operators to work with strings
- Plus [+] **concatenates** or "pastes" strings together
 - `String s1 = "How", s2 = "now";`
`String s3 = s1 + " " + s2;`
- You can also use +=, but only on a string variable
 - `s3 += " brown cow";`
- Exercise 5:** Use CodePad to concatenate your name and the null string. Screen-shot of the result.

Retrieving Characters

- Characters stored "contiguous" in memory
 - 
- You can access each character by its **position**
 - Note, though, that numbering starts at 0, not 1
- To retrieve a character, use **charAt(int position)**
 - Value returned is a **char**, not a **String**

Using charAt()

- Here are some examples that use **charAt()**
 - 1: `String s1 = "Conch", s2 = "fritters", s3 = "";`
 - 2: `char firstChar = s1.charAt(0);`
 - 3: `char lastChar = s2.charAt(s2.length()-1);`
 - 4: `char whatChar = s3.charAt(0);`
 - 5: `char thatChar = "What".charAt(0);`
- Exercise 6:** Use CodePad to try out each of these examples. Show me a screen-shot of the result.

Searching Methods

- Return the **position** of a character or string
 - `indexOf(arg)` returns position of the first occurrence
 - `lastIndexOf(arg)` returns the position of the last
 - Both return -1 if arg can't be found in the search string

```
1: String s1 = "Abadabadoo";
2: int p1 = s1.indexOf("daba");
3: int p2 = s1.indexOf('o');
4: int p3 = s1.lastIndexOf('o');
5: int p4 = s1.indexOf("doom");
```

- Exercise 7:** Use CodePad to try out each of these examples. Show me a screen-shot of the result

Builder Methods

- Strings** are **immutable**, they can't be changed; **Builder** methods create **new Strings**, transforming existing ones
- `s.toUpperCase()`, `s.toLowerCase()`
 - Returns **String** translated to either upper or lower case

```
String upperPet = "pet".toUpperCase();
String lowerPet = "Pet".toLowerCase();
upperPet.toLowerCase(); // logical error?
```

- Exercise 8:** Use CodePad to try out each of these examples. Show me a screen-shot of the result. Why is the last one a logical error?

Replacing Characters

- Replace characters using `s.replace('a','b')`
 - Returns **String** with all occurrences of a replaced by b

```
String mod = "Mom".replace('m', 'd');
String bad = "Dad".replace('D', 'B');
```

- Java 5 and 6 **replace()** method which with the **CharSequence** interface, which includes **String**
- Exercise 9:** Use CodePad to try out each of these examples. Show me a screen-shot of the result

Substrings

- Substring methods **extract** a portion of a string, given a starting index and an (optional) ending index
 - `s.substring(start)`, `s.substring(start,end)`
- Version 2 returns substring starting (and **including**) position **start**, up to, and **excluding**, position **end**

```
1: String s1 = "Hello, World!";  
2: String s2 = s1.substring(7);      // World!  
3: String s3 = s1.substring(7, 12); // World!
```

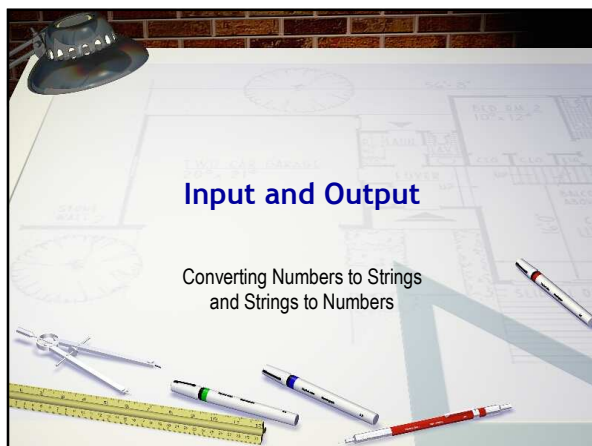


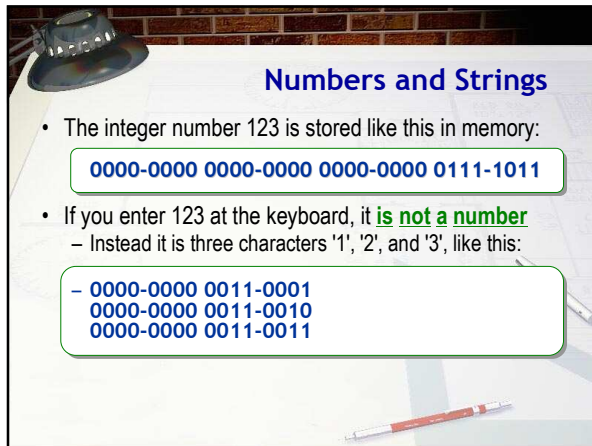
Express Yourself

- Exercise 10:** The string **name** contains "Last, First". You can use the **String trim()** method to remove extra spaces before and at the end of a string. You know that name contains a comma, but you aren't sure if there's a space after the comma or not. Using CodePad, initialize the variable name using your own name. Then, use the **indexOf()**, **substring()** and **trim()** functions to extract the first and last name into two new variables: **first** and **last**. Try your code both with and without extra spaces surrounding the comma. Show me a screen-shot of the result

Input and Output

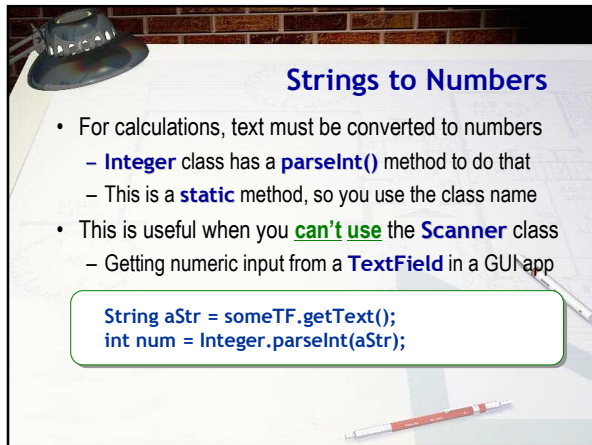
Converting Numbers to Strings
and Strings to Numbers





Numbers and Strings

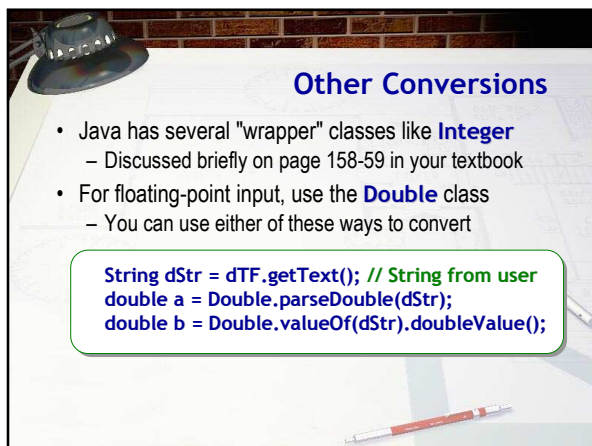
- The integer number 123 is stored like this in memory:
`0000-0000 0000-0000 0000-0000 0111-1011`
- If you enter 123 at the keyboard, it **is not a number**
 - Instead it is three characters '1', '2', and '3', like this:
 - `0000-0000 0011-0001`
 - `0000-0000 0011-0010`
 - `0000-0000 0011-0011`



Strings to Numbers

- For calculations, text must be converted to numbers
 - `Integer` class has a `parseInt()` method to do that
 - This is a **static** method, so you use the class name
- This is useful when you **can't use** the `Scanner` class
 - Getting numeric input from a `TextField` in a GUI app

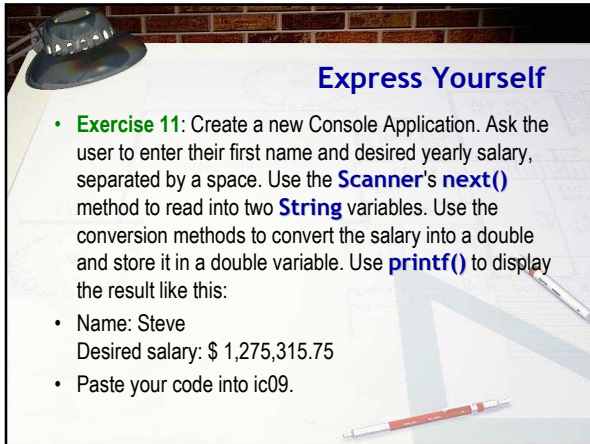
```
String aStr = someTF.getText();  
int num = Integer.parseInt(aStr);
```



Other Conversions

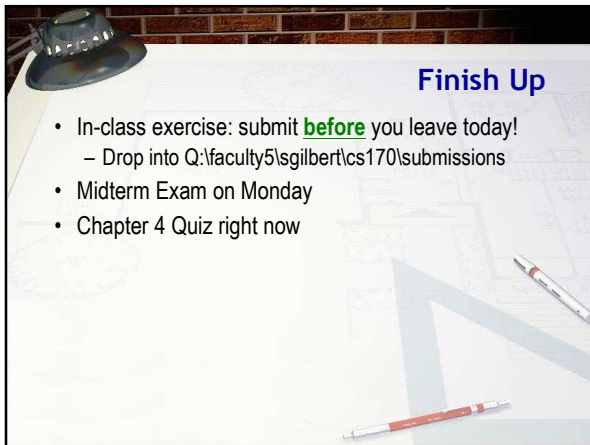
- Java has several "wrapper" classes like `Integer`
 - Discussed briefly on page 158-59 in your textbook
- For floating-point input, use the `Double` class
 - You can use either of these ways to convert

```
String dStr = dTF.getText(); // String from user  
double a = Double.parseDouble(dStr);  
double b = Double.valueOf(dStr).doubleValue();
```



Express Yourself

- **Exercise 11:** Create a new Console Application. Ask the user to enter their first name and desired yearly salary, separated by a space. Use the **Scanner's next()** method to read into two **String** variables. Use the conversion methods to convert the salary into a double and store it in a double variable. Use **printf()** to display the result like this:
 - Name: Steve
 - Desired salary: \$ 1,275,315.75
- Paste your code into ic09.



Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Midterm Exam on Monday
- Chapter 4 Quiz right now
